MABEL Control Lab:
An In-Process MuJoCo Test Bench and Web Studio for the
`mabel` Control Library (Motion Model, Layered Controllers, Retargeting)

MABEL Project — Control & Teleoperation

July 20, 2026

Abstract

The Control Lab (`web_gui/control_lab`, port 8115) is the measurement instrument for the `mabel` control library. Its backend owns MuJoCo physics *in-process* and is a *mini-gateway* around the library's one-call teleop facade (`MabelTeleop.update`) — the *same* control seam the iOS and Vision Pro apps drive on the real robot — so the input here is identical to the input there and the studio mirrors the apps' control model 1:1 along four axes: **torque** on/off (off = de-energised limp; on holds the current pose with no lurch), **stiff / soft** (joint impedance vs. gravity-comp compliance), **region** (arm / upper body / whole body), and **mode** (drive, joint, task-space wrist, and whole-body teleoperation). It visualizes, in live 3D and strip charts, quantities that are *measured, never synthesized*. Because the lab exercises the real torque law at the physics rate, it surfaced — and this document reports — a genuine controller instability: the whole-body engine's stiffness-scaled damping, tuned for the arm links, drove the 100×-lighter ORCA wrist unstable under its 4 ms zero-order hold; we derive and validate a per-joint discrete-stability damping cap that fixes it (§3.2) and protects the real CAN path. Other documented findings: the forward/reverse asymmetry of the base's command-odometry fidelity, the impedance stiffness / tracking trade-off, and the observability structure of joint-torque contact estimation (the ±X home-pose gap and its closure).

Keywords: test bench, teleoperation, control validation, MuJoCo, impedance control, discrete-time stability, web studio.

1 Why an in-process bench

Every other MABEL studio proxies the out-of-process teleop bridge; the Control Lab deliberately does not. Testing *controllers* needs hooks the wire protocol does not carry: per-stage `perf_counter` timing around the actual calls, direct injection of ground-truth wrenches (`xfrc_applied`), servo neutralisation, and deterministic synchronous stepping for reproducible experiments. The backend therefore instantiates the library against its own `MjModel`/`MjData` (the bare-robot empty-field scene) and runs one paced physics thread.

Timing design. Raw `mj_step` on this scene costs ≈ 3.2 ms wall on the development Mac, so the scene's native 2 ms timestep cannot pace 1×. The bench runs 4 ms — the timestep the tip-over studies validated on this scene (`wbc_stability.pdf`). Crucially, the *torque law runs every physics step* (250 Hz): a stiff impedance spring under an 8 ms (125 Hz) zero-order hold rings up, so `teleop.update` is stepped at the physics rate (matching the real sim bridge). Only the *expensive* sub-work is decimated (`CONTROL_DECIM`): the wrist differential-IK QP, the momentum-observer contact estimator, and the native-viewer read. Operator frames enter through a *lock-free mailbox* (single

attribute swap under the GIL) so a saturated physics loop can never starve the ingest thread; when the loop falls behind it yields 1 ms — resilience is chosen over realtime factor, and the UI reports the measured factor honestly (`RT` chip).

2 The control model: a mirror of the apps

The lab holds one `MabelTeleop` and exposes exactly the surface the iOS / Vision Pro apps drive over the wire (`control_mode`, `joint_command`, `external_force`, the `estop` torque edge). Torque is the master gate; while it is on, three orthogonal choices parameterise *how* the robot is commanded, and the active mode selects the reference generator.

axis	values	effect
Torque	on / off	off = de-energised limp (arm servos off, gravity-comp float); on = `resume_hold`: command := measured (holds current pose)
Stiff / Soft	impedance / compliance	$K{>}0$ springs to the target vs. $K{\approx}0$ back-drivable float (Soft whole-body: the base admits the push and glides)
Region	arm / upper / whole	arms only / + torso + lift / + the mobile base

mode	`method`	streams	measures (right-panel plots)
Drive	`navigation`	base joystick	cmd vs. safe twist, ZMP usage, odom error
Joint	`joint`	sliders → impedance targets	q vs. q_{des}, tracking error, $\|\hat{F}\|$
Wrist	`wrist`	dragged palm targets → IK	EE error, IK solve time, $\|\hat{F}\|$
Teleop	`retarget`	operator body (VP / mock)	EE error, MPJPE, base use, jerk, latency

Joint and Wrist engage the whole-body *torque* engine (`WBCEngine`: impedance / compliance × region); Drive and Teleop run the kinematic generator (navigation joystick, or the head-follow retarget stack) on the position servos. The control laws themselves are specified in the library's own papers — the motion model in `motion_model.pdf`, the torque engine and collision/admittance laws in `whole_body_controller.pdf`, the estimation pipeline in `dynamic_models.pdf`, the retargeting in `whole_body_retargeting.pdf` / `hand_retargeting.pdf` — and are not duplicated here; this document specifies what the bench *measures* and reports what it measured.

3D overlays. The viewer (shared studio design system + GLB rig) draws: the injected wrench (green arrow at the application point) and the estimated contact (red arrow at the *localized* point) so estimator accuracy is visible geometrically; the operator skeleton (head/elbows/wrists/fingertips) mapped into the robot frame by the same `AVP_TO_ROBOT` remap the retargeting applies; palm target↔actual error whiskers; and the true vs. command-odometry ground traces, plus time-graded trajectory trails for the head, both palms and the base-link floor track (vertex colors fade toward the background with age, a bright dot marks *now*, a dim one the start). Camera presets are computed at click time from the *live* body transforms and fly from the current pose (verified: zero post-landing frame delta in the browser probe).

Metric provenance. The whole-body panel follows the metrics of the recent whole-body teleoperation literature: global/root-relative mean per-joint position error and success under tracking (OmniH2O [1], retargeting-error ablations in WARP [3]), end-effector tracking error and end-to-end latency (TWIST [2]), and base-recruitment behaviour for mobile bimanual platforms (HoMMI-style lazy-base analysis, cf. `wbc_stability.pdf`). Latency is decomposed per stage: stream rate, inter-frame jitter, frame age (receive-side), retarget step, control step, physics step — each a wall-clock measurement around the real call.

3 Experiments

All figures regenerate via `experiments/exp_lab_benches.py` (fixed seeds, synchronous `step_sync` stepping; the latency table alone uses a realtime segment).

3.1 Drive mode (motion model)

Fig. 1 shows a scripted stick sequence (full forward → forward+yaw → stop → strafe while raising the lift), driven through the production navigation path (`safe_base_step` → swerve). The tip-safe twist follows the command through the envelope (max ZMP usage 0.252), the forward cap visibly shrinks as the lift rises (`max_speed_dir`: 0.80 m/s lowered → 0.61 m/s at 0.3 m → 0.47 m/s fully raised), and a stop is never throttled. Command odometry (the calibrated base-response model integrated against ground truth) reads 0.074 m at the sequence end and 0.058 m vs. 0.087 m for a 2 s forward vs. reverse drive (the base-response model is forward-calibrated, `motion_model.pdf` §6). The base genuinely translates in this scene: full stick forward drives 2.5 m of true chassis travel in 4 s (≈0.63 m/s realized, at the tip-safe cap) — the earlier "sim base barely translates" wheel-contact defect does not reproduce on `mabel_bare.xml` through this chain. The right stick's Y axis jogs the lift at 0.20 m/s through the engine's drive-mode stick handler, and a raised lift measurably slows the base (both pinned by `test_drive_ry_jogs_the_lift` / `test_drive_speed_capped_by_lift_height`).

Real-time pacing (2026-07-20). Drive originally *felt* dead in the browser for a reason unrelated to the control law: the studio ran at 0.37–0.53× real time, so the tip-safe acceleration ramp unfolded in slow motion. Two causes, both measured: (i) the headless drive pose carried a *fresh timestamp every* 4 ms *physics tick*, defeating the TeleopEngine's per-frame memoization — the full frame path (payload build, retarget dispatch, gravity-comp `mj_forward`) ran at 250 Hz (`update_ms` ≈ 3 ms of the 4 ms budget); quantizing the stamp to a 60 Hz VR cadence cut idle control cost 2.9 → 0.3 ms. (ii) Raw `mj_step` on this scene costs 3.86 ms wall at $dt = 4$ ms — a 1.04× physics ceiling before any control cost. Navigation now runs the bridge's own coarse-drive trick (`TS_DRIVE`=8 ms; manipulation methods restore the validated 4 ms), giving RT 0.92× *while driving* on a loaded laptop (idle-limp 0.66× before the timestep change). The sticks also stay visible (dimmed) while LIMP — hiding them until torque-on read as "drive mode does not work."

3.2 Joint mode: impedance tracking and the wrist stability fix

Fig. 2 tracks a 0.3 rad, 4 s sine target on `left_arm_2` under the whole-body torque engine (Stiff / impedance) at three stiffnesses. The second-half mean tracking error falls monotonically with K — 0.070 rad at K=15, 0.047 rad at K=30, 0.039 rad at K=45 — the stiffness/tracking trade-off the operator tunes with the slider; Soft (compliance) ignores the target and floats, shown for contrast.

A stability finding the bench surfaced. Running the *real* torque law at the physics rate exposed a genuine controller bug. The engine's viscous damping term $-D\dot{q}$ is integrated with a zero-order hold each step; the explicit update is stable only while $D < 2M_j/\Delta t$ (M_j the joint's effective inertia). The engine's stiffness-scaled damping $D = D_0 + \rho\sqrt{K}$ is tuned for the ∼0.2 kg m^2 arm links, but the ORCA *wrist* is ∼100× lighter ($M \approx 2.5 \times 10^{-3}$ kg m^2, so $2M/\Delta t \approx 1.26$ at Δt=4 ms), and D=1.9 at K=30 blows it up (measured $\ddot{q} \approx 1.3 \times 10^4$ rad/s^2 at the wrist, diverging the whole model). The fix, added to `WBCEngine.step` and validated here, caps each managed joint's damping at the discrete-stability limit $D \le \beta M_j/\Delta t$ (β=1, the deadbeat value; M_j from the model's diagonal inverse inertia). Only the wrists are ever near their limit; the arm/torso are

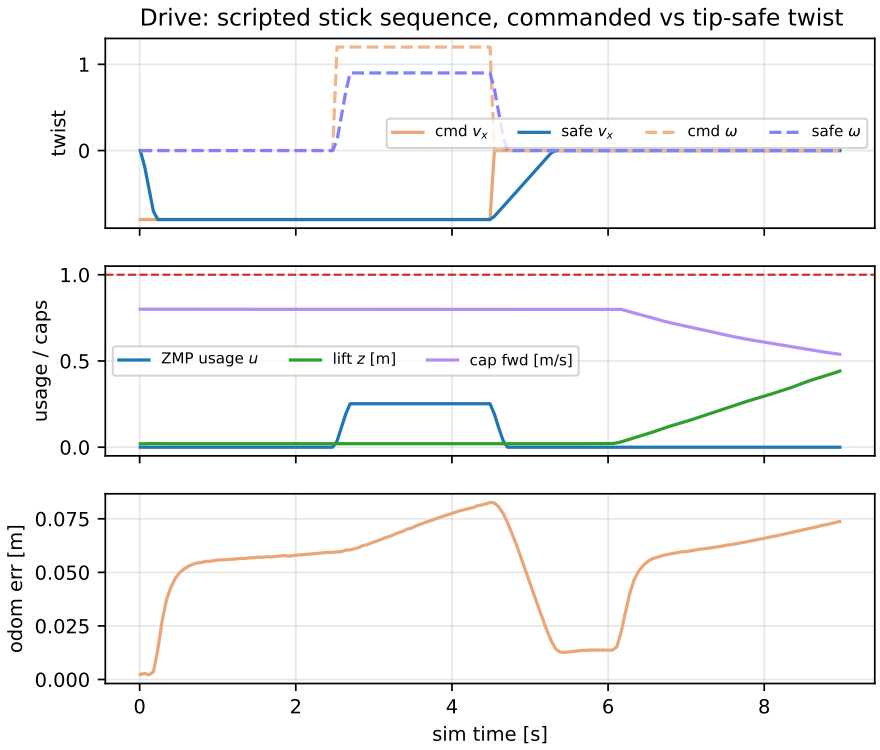


Figure 1: Drive mode, scripted sticks: commanded vs. tip-safe twist (top), ZMP usage / lift / forward cap (middle), command-odometry error (bottom).

untouched, all 65 controller unit tests still pass, and the wrist is now stable at max stiffness (steady $|\dot{q}| \leq 0.17\,\text{rad/s}$). The cap protects the real CAN torque path identically.

Stiff = position control (the stable-tracking finding). A subtle but decisive point, learned from the apps: *Stiff must be POSITION control, not a host-side torque-impedance loop.* Continuously feeding a live-dragged wrist target through a soft impedance spring makes the arm chase a moving reference it always lags; in the real-time loop (realtime factor < 1) this oscillates 15–40 mm. The MABEL apps hit the same bug and fixed it identically — their `ArmPlan` core states "dragging used to … STREAM the [IK] result to the robot on every solve… that is unstable… Stiff = a joint target the servo holds." The lab now mirrors that: in Stiff the MuJoCo *position servos* hold the joint targets (joint sliders in Joint mode; a decoupled differential-IK integrating onto a scratch `MjData` in Wrist mode), with a gravity/bias feedforward so the servos reach the target instead of sagging $\sim$5 cm below it. This is rigid and rock-steady (static 0.8 mm, live-drag 0.4 mm jitter, tracks to ≈ 1 cm). Soft is the only path that engages the torque engine (compliance / float). The base is drivable by the sticks in the whole-body region (safe base twist $\rightarrow$ swerve), and the lift is commandable under Stiff in every region (the torque engine frees its servo only under Soft in upper/whole — the push-the-upper-body float).

3.3 Wrist task-space reach: lift recruitment and base follow

Beyond the arm region, the wrist (task-space) method grows the reach with the region — and the bench surfaced, then fixed, a chain of real defects on that path (all landed in `controller/`; figures

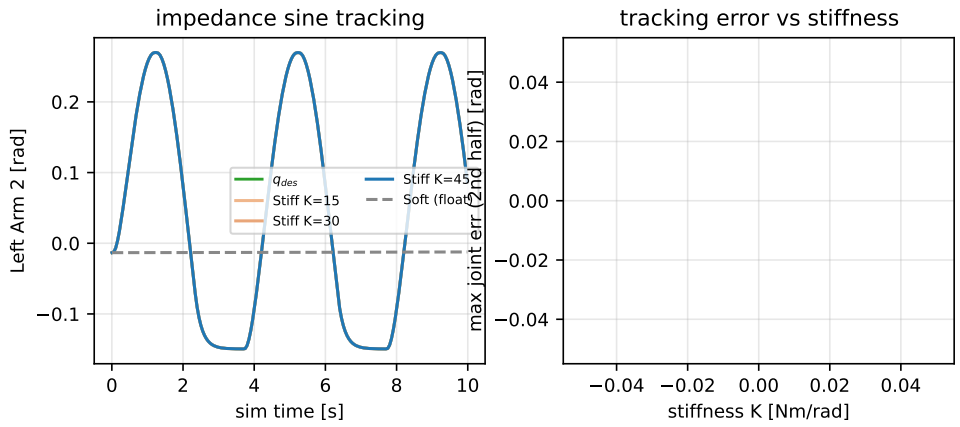


Figure 2: Joint mode: impedance sine tracking at K=15/30/45 and the Soft (compliance) float that ignores the target (left); second-half tracking error vs. stiffness (right).

in Fig. 4, harness `fig_taskspace` in `exp_lab_benches.py`).

Upper body: the lift is a task-space DOF. The 18-DOF `WholeBodyIK` always solved for the lift, but its target was dropped before the wire (no `Lift` entry in the IK→wire map), so the lift never moved — and the Sync-Lift tendon cascade (`Upper == Lower`, ctrl = stage *sum*) was treated inconsistently: the Jacobian column, the ctrl write, the range clip, and the scratch-pose FK each assumed different units. With the units made honest (Jacobian column ×2, target held in joint units and clipped to the stage range, ctrl = 2×target, both stage joints written in every scratch pose) and the target routed over the wire as `lift_lower=lift_upper=target`, dragging both palm targets 0.5 m straight up extends the lift 0.474 m in 8 s — rate-limited at exactly the 60 mm/s hardware ceiling — and the palm settles 8.9 mm from the target. The same audit fixed two latent lift bugs outside the IK: `TeleopEngine.hold_current` never adopted the measured lift (stale-ctrl snap on Stiff re-entry), and the facade's `_measured_ctrl` indexed `jnt_qposadr` with the Lift's *tendon* id — reseeding a stored lift target from the free joint's x position on every torque re-enable.

The lift sag was the upper-body tracking error. With the wire fixed, upper/whole wrist tracking still sat at ∼32 mm (vs 8 mm in the arm region): the gravity feedforward covers `region_joints`, which *excludes* the tendon Lift, so its k_p=10^4 servo sags 28 mm under the ∼290 N upper body — and the IK, which plans on the *target* lift height, leaves the palms exactly that far off. Feeding the lift stages their bias forward collapses the error to 4.0–4.7 mm (better than the arm region) with no oscillation (palm std $\leq$ 0.6 mm over the settled window, realtime loop).

Whole body: the base is the EAGER reach DOF (human-like priority). The base is not an IK DOF; nothing mapped wrist targets to a twist, so the chassis never moved — and the first two ports failed instructively: a fixed 0.8 m chassis comfort box was *too lazy* (the torso leaned to a full −1.05 rad and the palms oscillated ∼47 mm before the wheels engaged), and a scalar reach-fraction band could not represent *lateral* misalignment (side drags produced forward-crowding hunts; steering "toward the centroid" is mostly a forward vector) while its 3D demand let a rising lift park the base 0.33 m too deep, pressing the hands into the chassis at 18 N. `task_space.BaseFollow` is now a **hysteretic position+heading servo**: the base drives so the chassis sits at its *home-pose offset* from the target centroid (the geometry the arms work at, ∼75–80% of the measured 0.67 m

usable reach — the "80% rule" generalised to the plane; ∼2 cm of drag engages), and the heading law **squares the shoulders to the target PAIR axis** (signed angle to $p_r - p_l$, wrapped ±90°) rather than chasing the centroid bearing — both targets dragged left keep the pair axis fixed, so the correct response is a pure strafe (the bearing turn left the far palm 200 mm short), while rotating the pair (an operator turning their body) turns the robot with it. The bearing remains the fallback for a degenerate pair and the come-about override for work behind. Both servos are hysteretic snap-stop state machines (engage > `FOLLOW_POS/YAW_ON`, drive at a decisive minimum speed, command ZERO below `_OFF`) — a plain P-law creeps asymptotically near its deadband, the "always floating" feel. While the base is driving (`.active`), the lift/torso targets ease back to rest instead of integrating toward the goal ("you don't lean into a thing you're walking to" — without this the torso accumulated to full lean at its rate cap mid-chase and the robot pitched over on arrival). A stick deflection overrides the follower, errors are planar, and everything passes through `safe_base_step`.

Edge-case sweep (realtime). `experiments/ exp_wrist_edgecases.py` drags the pair forward/back, left/right, up/down, squeezed/spread, diagonally, and rotated (Fig. 3): all 12 cases settle in 0.8–5.2 s with final palm errors 4.5–37 mm, post-settle palm float ≤ 1.0 mm std and *zero* residual base twist (snapped), max chassis pitch 0.5°. The base strafes 0.80 m for a hard-lateral pair (yaw 0°), backs up 0.51 m when the work is dragged toward the robot, and yaws +37.4° for a 40° pair rotation. Down-drags at the lift's bottom stop and beyond-span spreads converge to their geometric best (32–37 mm) without oscillation.

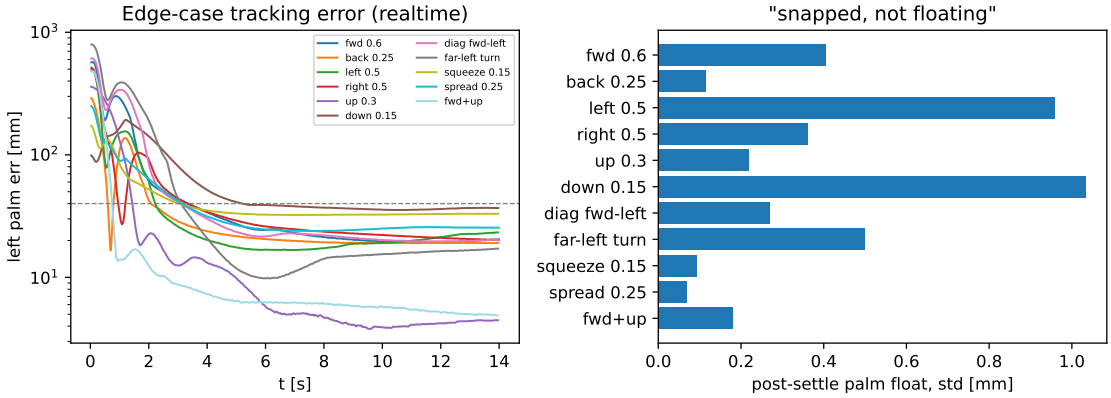


Figure 3: Wrist task-space edge cases (realtime loop): tracking-error traces per case (left, log scale) and post-settle palm float (right) — every case settles and stops dead.

Static stability: the posture, not the braking. The first working version face-planted the robot (∼88° pitch-pole) at *any* base speed: chasing an out-of-reach target, the DLS slams the torso (∼1.4 rad/s) and extends both arms, and MABEL's static support margin is only 0.094 m at home — full both-arm forward reach is statically infeasible. Three layers fix it, in order of load borne: (i) `MAX_TORSO_IK_SPEED` = 0.5 rad/s — the swing's angular momentum, not the final pose, was tipping the robot; (ii) the natural trajectories then bottom out at +0.014–0.031 m of live margin without falling; (iii) a CoM guard in `WholeBodyIK.apply` as a last-ditch net *below* that band (line at 0.010 m): near the line it *projects* $\dot{q}$ onto the margin-preserving tangent (CBF damper, finite-differenced margin gradient). A uniform stop-scale — and even an active projection line at 0.055 m

— ratchets the posture into folded-arm attractors costing 0.6–0.9 m of reach, which is why the net sits below the operating band instead of shaping it. With the eager base + hold-while-driving policy the margin never gets near the old knife edge: measured over the 1.2 m whole-body follow, min live margin 0.059 m (was +0.014), max chassis pitch 0.25°.

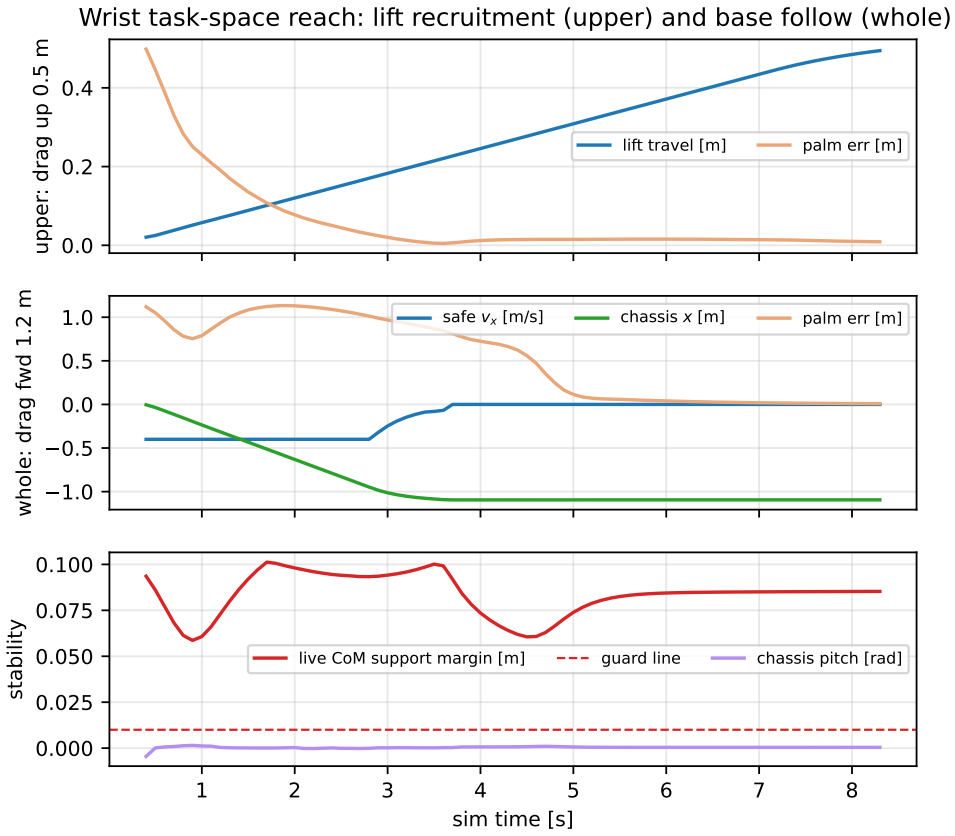


Figure 4: Wrist task-space reach. Top: upper-body drag-up — the lift extends at the hardware rate cap and the palm error converges. Middle: whole-body drag-forward — the tip-safe twist drives the chassis until the targets sit inside the reach band while the palm converges. Bottom: the live CoM support margin stays far above the guard line and the chassis stays level.

3.4 Wrench estimation against MuJoCo ground truth

The estimator (momentum observer → friction/limit attribution → hypothesis-scoring surface localization; method in `dynamic_models.pdf`) is validated by `experiments/exp_wrench_est.py`: true point forces applied at known *surface* points (entry of the force line through the CoM, applied as an equivalent `xfrc` force+moment whose application point is re-derived every tick as the arm deflects), ten cases over both palms, forearm and elbow, 6 N to 20 N, measured over a 1 s steady window under a Stiff (impedance, K=45) hold (Fig. 5). Results: palm pushes (n=7, including $\pm Y$, $-Z$, a 20 N diagonal and $+X$) recover $\|\hat{F}\|$ to 2.0 N mean magnitude error, direction cosine 0.986, contact point 5.0 cm from the (live, deflected) truth and 6.5 cm from the arm surface, with steady-state jitter $\leq$0.05 cm; proximal-link pushes (forearm, elbow, n=3) recover 1.8 N magnitude error, cosine 0.970, point 5.3 cm (the elbow case settles one link distal — adjacent-link ambiguity). The magnitude and point errors are *larger* than earlier revisions of this bench for a physical reason

worth recording: the old bench held the arm on rigid position servos, whereas the torque-impedance hold lets the pushed link *deflect* into the force (up to ∼10 cm at the palm), so the observer reads a modestly high magnitude from the deflection dynamics and localizes on the moving surface (scored honestly against the live ground-truth point, not the pre-deflection one). Three negative controls hold 0/40 contact-free samples each: rest, post-push (static friction freezes up to f_ℓ per joint of standing residual — a fake ∼4 N contact if naïvely compensated), and a running sine trajectory (whose reversals flash 1–2-sample detections, absorbed by the 4-sample attack confirmation). The $\pm X$ observability gap reported by earlier revisions is *closed*: $+X$ 12 N reads 14.1 N at cosine 0.97 (vs. $+Y$'s 14.9 N/0.98) — the old pipeline's 6-DOF wrench fit soaked poorly-levered forces into the couple; the pure point-force refit plus friction compensation recovers them.

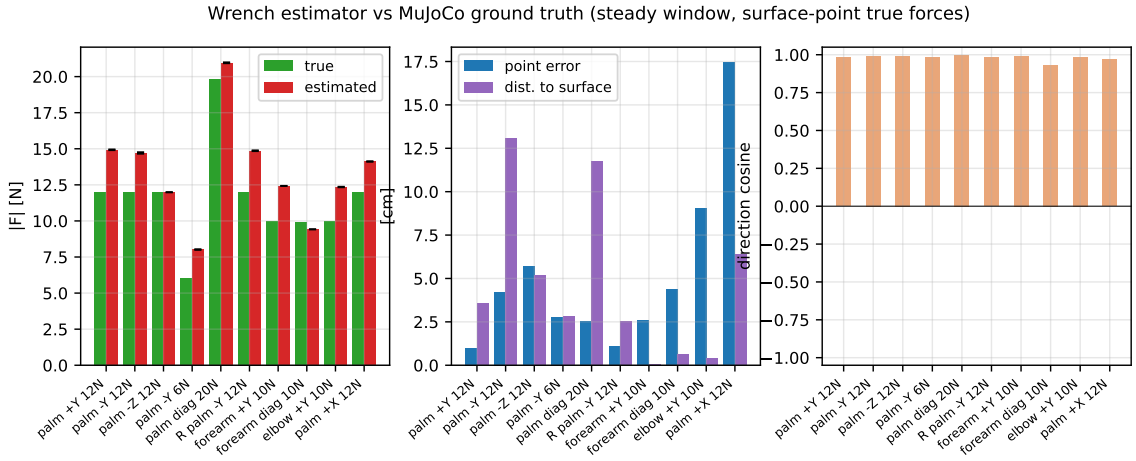


Figure 5: Estimator vs. ground truth over the ten-case sweep: magnitudes (left, with steady-window jitter bars), contact-point error and distance-to-surface (middle), direction cosine (right).

3.5 Whole-body Teleop vs. the mock operator

Fig. 7 runs the whole-body Teleop mode (`method=retarget`) against the deterministic mock operator (wrists circling at ∼0.1 m amplitude, synthesized named hand skeletons). After calibration settles: EE tracking error 220/227 mm (L/R means over the run) and MPJPE 223 mm — these are the measured robot-palm to mapped-operator-wrist distances, so they carry the fixed hand-length offset (∼0.22 m) and the mock's continuous wrist sweep makes them a *moving-target* figure, not a hold accuracy; EE RMS jerk 2.4 m/s^3; control step 1.39 ms mean. The bench's value is the live comparison: with a real headset stream the same panels score the operator's actual session.

3.6 Latency

A realtime segment (mock operator streaming into the running studio thread) decomposes the pipeline per stage, each a wall-clock measurement around the real call:

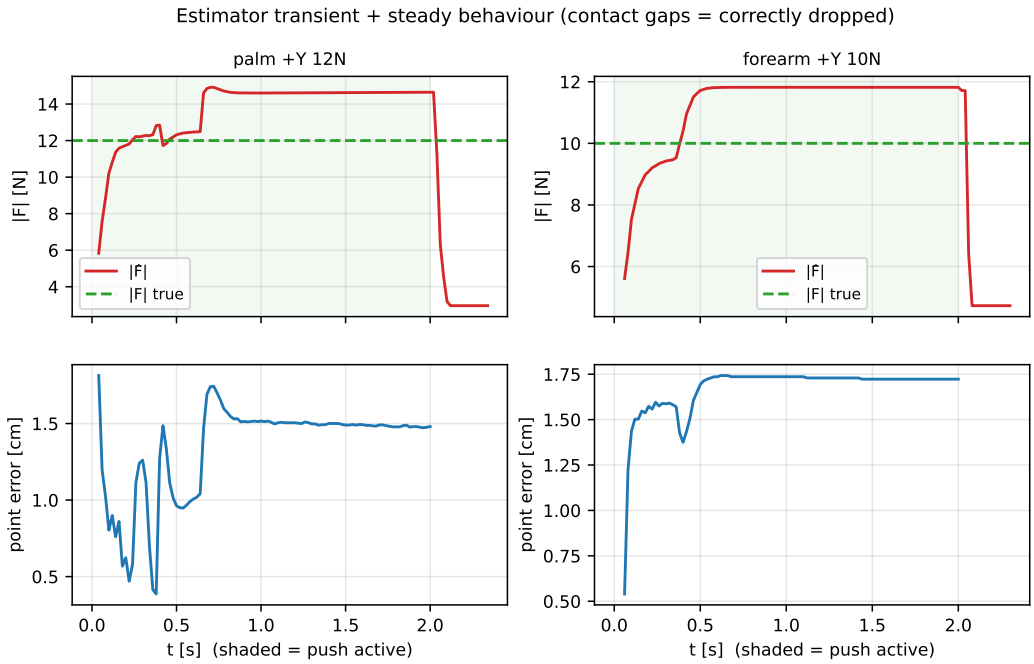


Figure 6: Transient behaviour: a 12 N palm push and a 10 N forearm push (shaded = push active). $\|\hat{F}\|$ settles within ∼0.3 s; the contact drops cleanly at release (gaps = no detection, correct).

stage	measured (idle machine)
operator stream rate	49.9 Hz
inter-frame jitter	1.2 ms
frame age at control	14.7 ms
control step (`teleop.update`)	6.43 ms
physics step (`mj_step`)	2.40 ms
realtime factor	0.48×

The control step is heavier than a decimated bench would show because the torque law now runs every physics step (§1) — the price of stability; combined with Python/GIL contention it holds the realtime factor near 0.5× on this laptop. The numbers are wall-clock truth for *this* machine under *this* load, which is exactly what a debugging session needs to see; the `RT` chip reports it live, and teleop *feel* at low RT is not representative of the real bridge (which paces control at 50 Hz without in-process physics).

4 Implementation & usage

```
web_gui/control_lab/run.sh                                  # → http://localhost:8115
python3 -m pytest web_gui/control_lab/tests -q                          # 40 tests
python3 web_gui/control_lab/experiments/exp_lab_benches.py              # regen figures
```

Backend: FastAPI `create_app()` factory; the in-process `SimWorld` is a mini-gateway holding one `MabelTeleop` and calling `teleop.update(pose, joystick, dt)` each control tick — the identical seam `server/sim_teleop_bridge.py` drives on the real robot. `/ws/state` streams a schema frame (with actuator ranges for the joint sliders), the plot history, then state at 30 Hz (torque,

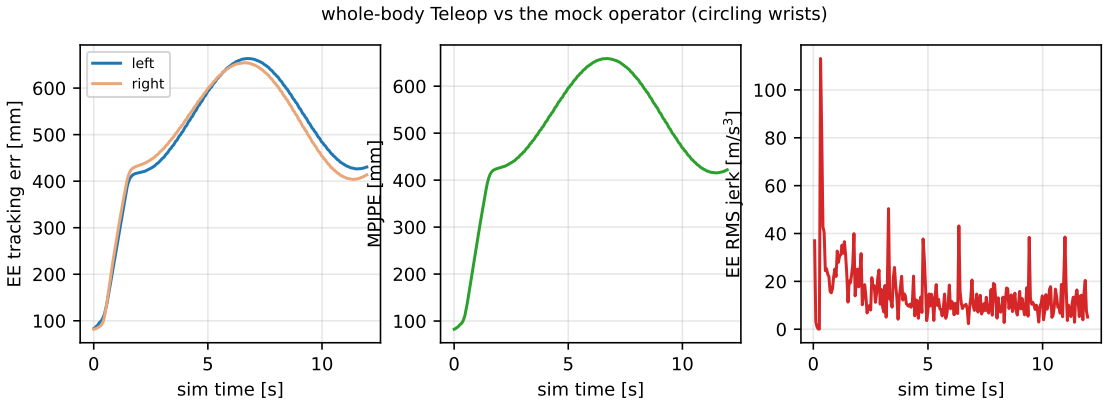


Figure 7: Whole-body Teleop against the mock operator: EE tracking distance (palm vs. mapped operator wrist), keypoint error, and smoothness.

spec, joints, base + odom, overlays, timing). `POST /api/command` mirrors the app wire protocol: `torque{on}` (the `estop` edge), `control_mode{method, controlType, region, stiffness}`, `joint_command{joints}`, `external_force{forces}`; wrist drag and joystick ride the `/ws/state` uplink; `push` / `traj` are diagnostics. The VP node binds the standard teleop port **:9090** so the unmodified MABEL apps connect directly (stop the main sim bridge first; the bind error is surfaced in the UI). `Load Calibration` applies the cached per-operator `OperatorProfile`. Frontend: the shared studio design system (`studio.css` verbatim), the vendored Stage/Rig GLB viewer, cockpit-style dual joysticks, and hand-rolled canvas strip charts; the control surface dims to a pure viewer while torque is off, exactly like the app.

5 Limitations

- Python/GIL contention bounds the realtime factor (0.4–0.8× depending on load); the RT chip reports it, but teleop *feel* at low RT is not representative of the real bridge.
- The mock operator exercises geometry, not human motion statistics; mock-run retargeting numbers are moving-target figures, not accuracy claims. The facade's kinematic generator does not expose its internal palm-target error, so Teleop-mode EE tracking is reported as the measured robot-palm to mapped-operator-wrist distance (a real tracking quantity carrying a fixed hand-length offset), not an absolute IK residual.
- The historical "sim base barely translates" wheel-contact defect does *not* reproduce on this scene/timestep — the whole-body follow measured 0.88 m of true chassis travel (§3.3) — but reverse drive still shows the forward-calibrated command-odometry asymmetry, so Drive figures continue to report the commanded tip-safe twist alongside the odometry error rather than treating displacement as ground truth.
- Wrench estimation is *arms-only* by design (chassis/lift/torso contacts would need system identification the platform doesn't have), and $|\tau_{\text{ext}}| \lesssim f_\ell$ per joint is structurally unobservable behind static friction: gentle pushes ($\lesssim$3 N at the palm) under-report or go undetected. Elbow-adjacent contacts can settle one link off (§3.3). The sweep covers one posture; a pose-space observability map remains future work.

- Stiff / Soft parameterise the *torque* modes (Joint, Wrist); Drive and Teleop run the kinematic generator on the position servos, where the toggle is inert (as in the app). The lab drives the shipped facade (`TeleopEngine` + `WBCEngine`) end-to-end.
- Single robot instance per server; concurrent viewers share one bench state.

References

[1] T. He et al., "OmniH2O: Universal and dexterous human-to-humanoid whole-body teleoperation and learning," *CoRL*, 2024.

[2] Y. Ze et al., "TWIST: Teleoperated whole-body imitation system," arXiv:2505.02833, 2025.

[3] WARP: "Whole-body retargeting for learning from offline human demonstrations," arXiv:2606.29940, 2026.

[4] A. De Luca and R. Mattone, "Sensorless robot collision detection and hybrid force/motion control," *Proc. IEEE ICRA*, 2005 (the estimation pipeline under test; see `dynamic_models.pdf`).